



Modules in Ansible

Understanding Configuration Management & Automation

Introduction

In Ansible, **Modules** are **predefined scripts** that perform specific automation tasks such as installing software, managing users, copying files, and configuring networks.

Ansible Modules help automate tasks **without writing complex scripts**. They allow users to run commands across multiple servers efficiently.

This guide will cover:

- **What are Modules in Ansible?**
 - **Types of Ansible Modules**
 - **How to use Modules in Playbooks and Ad-hoc commands**
 - **Common issues and troubleshooting**
-

What are Modules in Ansible?

Ansible **Modules** are like **functions in programming** that execute specific tasks on remote servers. They help:

- Install software
- Start/stop services
- Manage users and files
- Configure networking
- Handle cloud resources (AWS, Azure, GCP)



*Think of Ansible Modules as **building blocks** for automation tasks!*



Types of Ansible Modules

Ansible provides **thousands of built-in modules** grouped into different categories:

Module Type	Purpose
Package Modules	Install/uninstall software (e.g., apt , yum , dnf)
Service Modules	Start, stop, restart services (e.g., service , systemd)
User Modules	Create/manage users (e.g., user , group)
File Modules	Manage files, directories, permissions (e.g., file , copy)
Networking Modules	Configure network settings (e.g., iptables , firewalld)
Cloud Modules	Manage AWS, Azure, GCP resources (e.g., ec2_instance , azure_rm)

💡 Modules make it easy to automate different aspects of system administration!

Using Ansible Modules (Ad-hoc Commands)

Modules can be used **without writing a Playbook**, using **ad-hoc commands**.

1. Check Server Connectivity (**ping** Module)

```
ansible all -m ping
```



2. Install Apache (**apt** Module)

```
ansible webservers -m apt -a "name=apache2 state=present" --become
```

3. Restart a Service (**service** Module)

```
ansible webservers -m service -a "name=apache2 state=restarted" --become
```



Ad-hoc commands are useful for quick tasks without writing Playbooks!

Using Modules in Playbooks

Modules are commonly used in **Playbooks** to automate multiple tasks.

Example: Playbook Using Modules (**webserver.yml**)

```
---
- name: Configure Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Install Apache Web Server
      apt:
        name: apache2
        state: present

    - name: Start Apache Service
      service:
        name: apache2
```



```
state: started
```

```
- name: Create a Web Page
```

```
copy:
```

```
content: "Welcome to my Ansible Managed Server!"
```

```
dest: /var/www/html/index.html
```

Run the Playbook:

```
ansible-playbook -i inventory.ini webserver.yml
```



This Playbook installs Apache, starts the service, and creates a homepage!

Working with File Modules

File modules allow users to **create, delete, and modify files and directories.**

1. Create a Directory (file Module)

```
tasks:
```

```
- name: Create a directory
```

```
file:
```

```
path: /home/ansible/logs
```

```
state: directory
```

```
mode: '0755'
```

2. Copy a File (copy Module)

```
tasks:
```

```
- name: Copy a file to remote server
```



copy:

```
src: /local/path/index.html  
dest: /var/www/html/index.html  
mode: '0644'
```

💡 *File modules help manage system files and directories efficiently!*

Managing Users with Modules

User modules help **create, modify, or remove users and groups**.

1. Create a New User (**user** Module)

tasks:

- name: Add a new user

user:

```
name: devops  
shell: /bin/bash  
state: present
```

2. Delete a User

tasks:

- name: Remove a user

user:

```
name: devops  
state: absent
```

💡 *User modules simplify user account management across multiple servers!*



Automating Cloud Resources with Modules

Cloud modules help automate **AWS, Azure, and Google Cloud** services.

Example: Launch an AWS EC2 Instance (**ec2_instance** Module)

tasks:

- name: Create an EC2 instance

amazon.aws.ec2_instance:

name: my-instance

key_name: my-key

instance_type: t2.micro

image_id: ami-12345678

region: us-east-1



Cloud modules help provision cloud infrastructure dynamically!

Common Issues and Troubleshooting

Problem	Solution
Module not found	Ensure the module name is correct (ansible-doc -l for list)
Permission denied	Use --become for sudo/root privileges
SSH connection failure	Check SSH access to remote machines
Invalid YAML format	Ensure proper indentation in Playbooks



Know More (FAQs)

1. How do I list all available Ansible modules?

Run:

```
ansible-doc -l
```

2. How do I check what a module does?

Use:

```
ansible-doc apt
```

3. Can I write custom Ansible modules?

Yes! You can create **Python-based custom modules**.

4. What is the difference between an Ad-hoc command and a Playbook?

- **Ad-hoc commands** run **single module tasks** without saving them.
- **Playbooks** store multiple tasks in a structured format.

5. Can I use multiple modules in a single Playbook?

Yes! You can combine **any number of modules** in one Playbook.

Conclusion

Ansible **Modules** are the **core building blocks** of automation. They help **install software, manage services, configure users, handle files, and automate cloud resources**.

👉 Try using **Ad-hoc commands and Playbooks** to explore different **Ansible Modules**!